

NETWORKS 2 – TP01 REPORT

Overview

The project is a 2-player game of Infinite Pong. It follows the classical rules of Pong but runs until a player leaves.

The players control one paddle each. They must stop the ball from entering in their goal, but must make the ball enter the other player's goal.

Hitting a goal with the ball gives a point to the player opposite of the goal.

Code Architecture

Game

We mostly used OOP to create the game. We created a simple physics, collider, and shape system to simulate a pong match.

The physics system just checks if a physics object has collided with any collider that is present in the game.

Each collider is responsible for handling a collision with a physics object. For example, a paddle must handle being hit by the ball, and does so by sending it back where it came from (depending on the angle in which the ball hit).

A goal manager is responsible for handling goals and to keep track of scores.

The shape controller allows a user to take control of a shape. We use this to allow the player to control their paddle.

The game is displayed using pygame, and is only done in the client.

The server only handles logic.

Networking

We used UDP and a Server-Client Structure for the networking part of the game. Both client and server will attempt to send 60 packets per second.

We'll first explain the networking part of the client:

- To connect to a server, a client sends a special packet to said server. The client then waits 1 second to receive an approval by the server. It will do this 10 times, and if the client has not received approval by the server by the 10th time it does this, it will give up and assume that it cannot reach the server.
If the client attempts to join a server that is full, the server will send back a refusal packet, and the client will not be able to join the server.
- If the client successfully connects to the server, they will receive an ID back from the server, which will be the player that the client will be controlling (1 for the left player, 2 for the right one).
- The client will then wait for a packet from the server, which indicates that the game has started.
- On each frame, the client will search the packet buffer for the most recent packet received by the server. There are 2 possible packet types that the client can receive at this stage:

- A quit packet. This packet means that the other player has left, so the client will end the game view and print the final score on the console.
Note: This packet takes priority over all other packets. That is, if this packet is received, and is not the most recent packet, the game will end anyway!
- A game state packet. This packet contains information about the game state that the client will process, so that it is synchronized with the server.
 This game state packet contains information about where the ball is, the ball direction, both player positions, the score, and the packet number:
 - The client will not always set the ball position to the same position as the server's, some wiggle room is allowed as to give a smooth experience to the clients.
 - Since ball direction is a lot more crucial, we did not allow any wiggle room, but we did allow the client a small chance (in milliseconds) to synchronize with the server. If this is not done in time, the client will set both its ball position and direction to the same ones as the server.
 - The client is only authoritative on one thing, and that is its own player position. The client will never set its own player position to the one given by the server, but it will always set its opponents position to the one received by the server.
 - The client knows a goal has been scored when the score received by the server is different than the client's.
- If the client has stopped receiving packets from the server for more than 10 seconds, it will assume that it can no longer reach the server, so it will end the game view and print the final score on the console.
- At the end of each frame, the client will send a package to the server containing its ID, position, and packet number. The server will use this packet to synchronize player positions.

Now, for the server:

- The server will start by waiting for a join request from 2 clients. Depending on the order of them joining, they will receive either the "1" player ID or "2".
- Once 2 clients have joined, the game will start to be simulated.
- On each frame, the server will search the packet buffer for the 2 most recent packets received by each client. There are 3 possible packet types that the server can receive at this stage:
 - A quit packet. This means that one of the players has left, so the server will end the game.
Note: This packet takes priority over all other packets. That is, if this packet is received, and is not the most recent packet, the game will end anyway!
 - A player position packet. This packet will be used to know where both players are at all times.
 - A join packet. This means that a third client is trying to join the server, which is not possible. The server will respond by sending a refusal packet.
- If the server has stopped receiving packets from any client for more than 3 seconds, it will pause the game and wait until it has received a packet from the missing client/s.
- If the server has stopped receiving packets from any client for more than 10 seconds, it will assume that the client/s can no longer reach the server, and will end the game.
- At the end of each iteration, the server will send a packet to both clients containing the game state (as seen above).

User Manual

Files

The user needs to run the “pong_client.py” file to start a client, or/and the “pong_server.py” file to start a server.

Client Setup

Upon starting a client, the user must enter a valid server address. If successfully connected, the game should start once a second client connects.

In the game, the client uses “W” to move its paddle up and “S” to move it down.

If “Q” is pressed, the client disconnects from the server and awaits a new server address.

If “CTRL+Q” is pressed, the client quits the game entirely.

Server Setup

Upon starting a server, the user must then enter a valid IPV4 address as well as an unused port (usually from 49152 to 65535).